

## CSCE 645 GEOMETRIC MODELING · MODULE 11 · LECTURE NOTES

# Implicit Surfaces and Distance Fields

*Defining shape by a function instead of by primitives.*

## MODULE OUTCOMES

- Define signed distance fields and their properties.
- Compose shapes with CSG and smooth blending operators.
- Implement sphere tracing to render implicit surfaces.
- Extract a mesh with marching cubes and state its limitations.
- Choose between implicit and explicit for a given task.

## 1 Implicit surfaces and signed distance

An implicit surface is the zero set of a function:  $\{x : f(x) = 0\}$ , with  $f$  negative inside and positive outside. Any such  $f$  defines a surface, but a particular class of them carries much more information.

**$|f(x)|$  = distance from  $x$  to the nearest point of the surface**

A function with this property is a **signed distance field**. Two consequences follow immediately:

- $|\nabla f| = 1$  **everywhere** (the eikonal equation). Moving a small step changes the distance by exactly that step.
- $\nabla f$  is the **unit surface normal**. No averaging, no weighting choice — just a gradient, available analytically or by finite differences.

Contrast with a mesh, where normals must be averaged with a weighting scheme (Module 02) and distance queries require a spatial acceleration structure and a search.

### 1.1 Primitives

```
float sphere(vec3 p, float r) { return length(p) - r; }

float box(vec3 p, vec3 b) {
    vec3 q = abs(p) - b;
    return length(max(q, 0.0))           // outside distance
           + min(max(q.x, max(q.y, q.z)), 0.0); // inside distance
}

float torus(vec3 p, vec2 t) {
    return length(vec2(length(p.xz) - t.x, p.y)) - t.y;
}
```

Each is an exact closed-form distance. The box expression looks cryptic because it handles both cases in one branchless formula: outside the box, only the positive components of  $q$  contribute; inside, all components are negative and the largest (least negative) gives the distance to the nearest face.

### TRANSFORMS, AND WHAT BREAKS THE DISTANCE PROPERTY

To transform a primitive, transform the *query point* by the inverse:  $f(M^{-1}p)$ . This is a frequent source of bugs, because the intuition points the other way.

Rigid transforms preserve the distance property. **Non-uniform scale does not** — after scaling,  $f$  is a lower bound on the true distance rather than the distance itself.

A lower bound is still safe for sphere tracing (§3): you step less far than you could, so you take more steps but never overshoot. Knowing which operations preserve exactness and which only preserve the bound is the practical skill in SDF work.

## 2 Composition

| Operation             | Expression                                        | Exactness                                                               |
|-----------------------|---------------------------------------------------|-------------------------------------------------------------------------|
| Union                 | $\min(f_a, f_b)$                                  | Exact outside; a lower bound inside.                                    |
| Intersection          | $\max(f_a, f_b)$                                  | Exact inside; a lower bound outside.                                    |
| Difference            | $\max(f_a, -f_b)$                                 | Bound in general.                                                       |
| Offset / shell        | $f(x) - d$ , or $ f(x)  - d$                      | Exact. Offsetting a mesh self-intersects; here it is a subtraction.     |
| Infinite repetition   | $f(\text{mod}(p, c) - 0.5c)$                      | Exact for well-separated instances. Unbounded content in $O(1)$ memory. |
| Twist, bend, displace | $f(\text{warp}(p))$ , or $f(p) + \text{noise}(p)$ | Bound only — the warp changes the metric.                               |

Compare each row against doing the same thing to a triangle mesh (Module 01). Boolean operations on meshes require computing exact intersection curves, re-triangulating both surfaces along them, and resolving near-degenerate configurations — a genuinely difficult problem that commercial kernels still get wrong. Here it is min.

### 2.1 Smooth blending

```
float smin(float a, float b, float k) {
    float h = clamp(0.5 + 0.5 * (b - a) / k, 0.0, 1.0);
    return mix(b, a, h) - k * h * (1.0 - h);
}
```

**THE OPERATION WITH NO MESH EQUIVALENT**

A hard union leaves a crease where the two shapes meet. A smooth minimum produces a controllable fillet, with  $k$  setting the blend radius, from a single extra line.

There is no corresponding mesh operation. Filleting a mesh junction means constructing the blend geometry explicitly, which is what CAD fillet operations do — and they are notoriously fragile, failing on tight corners and self-intersections.

This is the capability that drives most procedural and demoscene modelling toward SDFs: organic-looking blends between primitives, for almost nothing.

### 3 Rendering by sphere tracing

```
float trace(vec3 ro, vec3 rd) {
    float t = 0.0;
    for (int i = 0; i < MAX_STEPS; ++i) {
        float d = scene(ro + rd * t);
        if (d < EPS) return t;           // hit
        if (t > FAR) break;              // miss
        t += d;                          // step by the distance
    }
    return -1.0;
}
```

#### WHY STEPPING BY D IS SAFE

$f(p)$  is the distance to the *nearest* surface point in any direction. So a sphere of radius  $f(p)$  centred at  $p$  is guaranteed empty, and advancing along the ray by  $f(p)$  cannot pass through any surface.

This is what the distance property buys. A function that only reported inside or outside would force small fixed steps, with the risk of stepping through thin geometry.

If  $f$  is a lower bound rather than an exact distance — after a warp or non-uniform scale — the algorithm still works, taking more steps than necessary but never overshooting. Hence the common practice of multiplying warped fields by a safety factor.

Normals come from the gradient, estimated by central differences with four or six extra evaluations of the scene function.

| Strength                                                                           | Limitation                                                                                                                                    |
|------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Exact surfaces at any resolution; no tessellation, no LOD, no popping.             | Every pixel evaluates the <i>entire</i> scene function. Cost scales with scene complexity, not with how much of the screen the object covers. |
| CSG and blending are nearly free.                                                  | Grazing rays take many small steps while running nearly parallel to a surface without hitting it — the pathological case.                     |
| Soft shadows and ambient occlusion fall out of the distance field almost for free. | Artist-authored detail is hard to express as a function.                                                                                      |
| Infinite repetition and procedural detail at no memory cost.                       | Texture mapping requires a parameterisation, which an implicit surface does not have.                                                         |

The complexity-not-coverage property is the decisive limitation: a scene of a thousand distinct objects requires evaluating a thousand primitives per pixel unless you build acceleration structures, at which point much of the simplicity is gone. This is why sphere tracing dominates procedural and demoscene rendering and does not replace rasterisation for authored content.

## 4 Converting to a mesh

**Marching cubes** samples  $f$  on a regular grid. Each cell has eight corners, each inside or outside, giving 256 configurations (15 up to symmetry). A lookup table gives the triangles for each, with vertices positioned along cell edges by linear interpolation of  $f$  between the corner values.

### MARCHING CUBES ROUNDS OFF SHARP FEATURES

A vertex can only be placed *on a grid edge*. The corner of a cube lies in the interior of a cell, so it cannot be represented and the extracted mesh rounds it off.

Refining the grid makes the rounding smaller and never removes it: at any resolution the corner is still a staircase of small facets. This is the characteristic 'melted' look of marching-cubes output on hard-surface shapes.

**Dual contouring** fixes it by placing one vertex *anywhere inside* each cell, chosen to minimise error against the gradient information sampled on the cell's edges. Sharp corners are recovered exactly, at the cost of needing gradients and of producing non-manifold output in some configurations.

Marching cubes also produces many poorly shaped triangles — slivers where the surface grazes a cell corner — so output is normally remeshed (Module 10) before use.

## 5 Choosing, and using both

| Favours implicit                                                   | Favours explicit mesh                                        |
|--------------------------------------------------------------------|--------------------------------------------------------------|
| Boolean operations, blending, offsetting.                          | Artist-authored detail and precise control.                  |
| Procedural, infinite, or very smooth content.                      | Texture mapping, which needs a parameterisation (Module 09). |
| Distance and collision queries.                                    | Rasterised rendering at scale.                               |
| Topology changes during simulation — merging and splitting fluids. | Skeletal animation and skinning.                             |
| Compact storage of smooth shapes.                                  | Predictable, bounded per-frame cost.                         |

**MODERN ENGINES USE BOTH, FOR DIFFERENT JOBS**

**Soft shadows** — march toward the light and track the closest approach; the penumbra falls out of the distance values.

**Ambient occlusion** — sample the field at a few points along the normal and compare against the distance travelled.

**Global illumination** — Unreal Engine's distance field GI traces coarse per-object SDFs rather than the triangle geometry.

**Collision** — penetration depth is  $f$  and the contact normal is  $\nabla f$ , both immediate.

**Text and decals** — signed distance textures give resolution-independent sharp edges under arbitrary magnification, which is why they replaced bitmap fonts in engines.

The division of labour is clean: **meshes render; distance fields answer questions about space.**

## Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

| Resource                                                                                | What it covers for this module                                                                                                                                     |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Inigo Quilez — distance functions and articles (free)</a>                   | The primary practical source: exact SDFs for dozens of primitives, blending operators, soft shadows, and ambient occlusion, all with derivations and working code. |
| <a href="#">Shadertoy</a>                                                               | Thousands of sphere-traced scenes with readable source. The fastest way to learn the idioms.                                                                       |
| <a href="#">Lorensen &amp; Cline — Marching Cubes (free summaries widely available)</a> | The original algorithm. The lookup-table construction is worth understanding once.                                                                                 |
| <a href="#">Ju et al. — Dual Contouring of Hermite Data (free)</a>                      | How to recover sharp features that marching cubes loses.                                                                                                           |

## Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a sphere tracer in a fragment shader. Render a sphere, a box, and a torus with gradient-based normals and simple lighting.
- 2 Implement union, intersection, and difference. Build a shape that would be genuinely painful to construct as a mesh, and note how long it took.

- 3 Implement  $s_{min}$  and sweep  $k$  from 0 to a large value, capturing the blend at several settings.
- 4 Implement infinite repetition with a modulo and render an unbounded field of objects. Confirm memory use does not depend on the number visible.
- 5 Implement soft shadows and ambient occlusion from the distance field. Compare the result and the cost against shadow mapping from CSCE 641 Module 10.
- 6 Apply a non-uniform scale to a primitive and demonstrate that the field is no longer an exact distance. Show that sphere tracing still terminates and count the extra steps.
- 7 Implement marching cubes and extract a mesh from an SDF of a cube. Capture the rounded corners at three grid resolutions and confirm they never become sharp.
- 8 Extract the same cube with dual contouring and compare.

## Self-check

---

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What distinguishes a signed distance field from an arbitrary implicit function, and what two properties follow?
- 2 Give the expressions for union, intersection, difference, and offset.
- 3 Which operations preserve exact distance and which leave only a bound? Why does the distinction not break sphere tracing?
- 4 Why is smooth blending the operation that most favours implicit representations?
- 5 Why is stepping by  $f(p)$  provably safe in sphere tracing?
- 6 What is sphere tracing's main scaling limitation, and why does it rule out large authored scenes?
- 7 Why does marching cubes round off sharp corners, and what does dual contouring change?